



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных
технологий (МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №4
по дисциплине
«Распределенные СУБД»

Выполнил студент группы ИКБО-66-23

Смирнов А.Ю.

Принял ст. преподаватель

Гуличева А.А.

Москва 2026

Денормализация

Сама по себе денормализация обладает одним достоинством, в отличие от обычного подход. Ее удобно использовать для хранения исторических данных, которые ни при каких обстоятельствах не должны изменяться. В качестве примера можно привести хранение покупок клиента со ссылками на товар. Т.к. цена товара меняется с течением времени, чтобы получить стоимость каждого товара на момент приобретения, нужно отдельно хранить цену товара. При использовании подхода с денормализацией, можно в таблице с покупками хранить полностью объект товара. При этом текущий товар можно изменять любым образом и даже удалять и это никак не повлияет на историю покупок клиента.

Запрос первичен

При проектировании, нужно учитывать, что запрос должен получить за один раз все данные из одной таблицы (т.к. нет join). Поэтому сначала проектируются все возможные запросы, а затем под них создаются таблицы. Таким образом в Cassandra создание структуры БД начинается с определения запросов.

Оптимальное хранение

В реляционных базах редко можно встретить рекомендации к структуре БД для оптимального хранения и чтения данных. Чаще всего никто не заботится об этом. В распределенных системах дело обстоит иначе, т.к. данные располагаются на нескольких узлах, более высокую производительность будет демонстрировать тот запрос, который отдает данные с одной ноды. Таким образом, желательно располагать данные так, чтобы данные возвращались из одной ноды.

Цель работы

Изучить принципы проектирования и работы с распределённой базой данных Apache Cassandra, а также получить практические навыки по развёртыванию кластера, созданию таблиц и выполнению запросов с учётом архитектурных особенностей этой СУБД..

Задачи:

1. Развернуть распределённый кластер Cassandra из трёх узлов с помощью Docker Compose, настроить дата-центры, тома и переменные окружения.
2. Понять концепцию первичного ключа в Cassandra, включая его разделение на ключ раздела (partition key) и ключи кластеризации (clustering keys), и осознать, как структура первичного ключа влияет на распределение данных по узлам кластера.
3. Освоить принцип денормализации данных и создания таблиц, ориентируясь на предполагаемые запросы, что является ключевым отличием Cassandra от реляционных баз данных.
4. Изучить механизмы обеспечения согласованности (consistency levels) и их влияние на скорость и надежность операций чтения и записи.
5. Познакомиться со стратегиями сжатия (compaction strategies) и сортировки данных внутри таблиц.
6. Проанализировать теоретические аспекты (выбор Snitch, формулы согласованности) и применить их на практике.

docker-compose.yml

Листинг 1 – docker-compose.yml

```
services:
  cassandra-seed-Smirnov:
    image: cassandra
    container_name: cassandra-seed-Smirnov
    environment:
      CASSANDRA_CLUSTER_NAME: "SmirnovAY"
      CASSANDRA_ENDPOINT_SNITCH: "GossipingPropertyFileSnitch"
      CASSANDRA_DC: "datacenter1"
      AUTO_BOOTSTRAP: "false"
      MAX_HEAP_SIZE: "2G"
      HEAP_NEWSIZE: "400M"
    volumes:
      - cassandra_data_seed:/var/lib/cassandra
    ports:
      - "19040:9042"
    mem_limit: 4g
    mem_reservation: 3g

  cassandra-node-Smirnov_1:
    image: cassandra
    container_name: cassandra-node-Smirnov_1
    environment:
      CASSANDRA_CLUSTER_NAME: "SmirnovAY"
      CASSANDRA_ENDPOINT_SNITCH: "GossipingPropertyFileSnitch"
      CASSANDRA_DC: "datacenter1"
      CASSANDRA_SEEDS: "cassandra-seed-Smirnov"
      MAX_HEAP_SIZE: "2G"
      HEAP_NEWSIZE: "400M"
    volumes:
      - cassandra_data_node_1:/var/lib/cassandra
    ports:
      - "19041:9042"
    depends_on:
      - cassandra-seed-Smirnov
    mem_limit: 4g
    mem_reservation: 3g
```

```
cassandra-node-Smirnov_2:
  image: cassandra
  container_name: cassandra-node-Smirnov_2
  environment:
    CASSANDRA_CLUSTER_NAME: "SmirnovAY"
    CASSANDRA_ENDPOINT_SNITCH: "GossipingPropertyFileSnitch"
    CASSANDRA_DC: "datacenter2"
    CASSANDRA_SEEDS: "cassandra-seed-Smirnov"
    MAX_HEAP_SIZE: "2G"
    HEAP_NEWSIZE: "400M"
  volumes:
    - cassandra_data_node_2:/var/lib/cassandra
  ports:
    - "19042:9042"
  depends_on:
    - cassandra-seed-Smirnov
  mem_limit: 4g
  mem_reservation: 3g
```

volumes:

```
cassandra_data_seed:
cassandra_data_node_1:
cassandra_data_node_2:
```

Описание выполнения задач

```
PS C:\Users\theso\Desktop> docker compose up -d
[+] up 7/7
✓Network desktop_default          Created      0.0s
✓Volume desktop_cassandra_data_node_2 Created      0.0s
✓Volume desktop_cassandra_data_seed Created      0.0s
✓Volume desktop_cassandra_data_node_1 Created      0.0s
✓Container cassandra-seed-Smirnov Created      0.1s
✓Container cassandra-node-Smirnov_2 Created      0.3s
✓Container cassandra-node-Smirnov_1 Created      0.3s
```

Рисунок 1 – Запуск docker compose для создания кластера

```
PS C:\Users\theso\Desktop> docker exec -it cassandra-seed-Smirnov cqlsh
Connected to SmirnovAV at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.6 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> |
cqlsh> CREATE KEYSPACE learn_cassandra WITH REPLICATION = {'class' : 'NetworkTopologyStrategy','datacenter1': 2,'datacenter2': 1};
cqlsh> CREATE TABLE learn_cassandra.users_by_country (
... country text,
... user_email text,
... first_name text,
... last_name text,
... age smallint,
... PRIMARY KEY ((country), user_email)
... );
cqlsh> |
```

Рисунок 2 – Создание пространства ключей и создание таблицы

users_by_country

```
cqlsh> INSERT INTO learn_cassandra.users_by_country (country,user_email,first_name,last_name,age)
... VALUES('UK', 'peter@email.com', 'Peter','Clark',65);
InvalidRequest: Error from server: code=2200 [Invalid query] message="keyspace learn_cassandra does not exist"
cqlsh> INSERT INTO learn_cassandra.users_by_country (country,user_email,first_name,last_name,age) VALUES('UK', 'peter@email.com', 'Peter','Clark',65);
cqlsh> INSERT INTO learn_cassandra.users_by_country (country,user_email,first_name,last_name,age)
... VALUES('UK', 'bob@email.com', 'Bob','Sandler',23);
cqlsh> INSERT INTO learn_cassandra.users_by_country (country,user_email,first_name,last_name,age)
... VALUES('UK', 'alice@email.com', 'Alice','Brown',26);
cqlsh> SELECT * FROM learn_cassandra.users_by_country;

country | user_email | age | first_name | last_name
-----+-----+---+-----+-----
US | john@email.com | 55 | John | Wick
UK | alice@email.com | 26 | Alice | Brown
UK | bob@email.com | 23 | Bob | Sandler
UK | peter@email.com | 65 | Peter | Clark

(4 rows)
cqlsh> |
```

Рисунок 3 – Заполнение таблицы

```

cqlsh> CREATE TABLE learn_cassandra.users_by_email (
... user_email text,
... country text,
... first_name text,
... last_name text,
... age smallint,
... PRIMARY KEY (user_email)
... );
cqlsh> INSERT INTO learn_cassandra.users_by_email (user_email, country, first_name, last_name, age)
... VALUES('john@email.com', 'US', 'John', 'Wick', 55);
cqlsh> INSERT INTO learn_cassandra.users_by_email (user_email, country, first_name, last_name, age)
... VALUES('peter@email.com', 'UK', 'Peter', 'Clark', 65);
... INSERT INTO learn_cassandra.users_by_email (user_email, country, first_name, last_name, age)
... VALUES('bob@email.com', 'UK', 'Bob', 'Sandler', 23);
Invalid syntax at char 12
VALUES('bob@email.com', 'UK', 'Bob', 'Sandler', 23);
cqlsh> INSERT INTO learn_cassandra.users_by_email (user_email, country, first_name, last_name, age)
... VALUES('bob@email.com', 'UK', 'Bob', 'Sandler', 23);
...
cqlsh> SELECT * FROM learn_cassandra.users_by_email;

user_email | age | country | first_name | last_name
-----+-----+-----+-----+-----
john@email.com | 55 | US | John | Wick
(1 rows)
cqlsh> INSERT INTO learn_cassandra.users_by_email (user_email, country, first_name, last_name, age)
... VALUES('peter@email.com', 'UK', 'Peter', 'Clark', 65);
cqlsh> INSERT INTO learn_cassandra.users_by_email (user_email, country, first_name, last_name, age)
... VALUES('bob@email.com', 'UK', 'Bob', 'Sandler', 23);
cqlsh> INSERT INTO learn_cassandra.users_by_email (user_email, country, first_name, last_name, age)
... VALUES('alice@email.com', 'UK', 'Alice', 'Brown', 26);
cqlsh> SELECT * FROM learn_cassandra.users_by_email;

user_email | age | country | first_name | last_name
-----+-----+-----+-----+-----
john@email.com | 55 | US | John | Wick
alice@email.com | 26 | UK | Alice | Brown
peter@email.com | 65 | UK | Peter | Clark
bob@email.com | 23 | UK | Bob | Sandler
(4 rows)
cqlsh> |

```

Рисунок 4 – Создание таблицы users_by_email и её заполнение

```

cqlsh> SELECT * FROM learn_cassandra.users_by_email WHERE age=26 ALLOW FILTERING;

user_email | age | country | first_name | last_name
-----+-----+-----+-----+-----
alice@email.com | 26 | UK | Alice | Brown
(1 rows)
cqlsh> |

```

Рисунок 5 – Фильтрация по столбцу, который не является ключом раздела

```

cqlsh> CONSISTENCY ALL;
Consistency level set to ALL.
cqlsh> SELECT * FROM learn_cassandra.users_by_country WHERE country='US';

country | user_email | age | first_name | last_name
-----+-----+-----+-----+-----
US | john@email.com | 55 | John | Wick
(1 rows)
cqlsh> |

```

Рисунок 6 – Установление уровня согласованности «ALL» с последующим получением данных

```
cqlsh> CONSISTENCY ONE;
Consistency level set to ONE.
cqlsh> SELECT * FROM learn_cassandra.users_by_country WHERE country='US';
```

country	user_email	age	first_name	last_name
US	john@email.com	55	John	Wick

```
(1 rows)
cqlsh> |
```

Рисунок 7 – Установление уровня согласованности «ONE» с последующим получением данных

```
cqlsh> DESCRIBE TABLE learn_cassandra.users_by_country;
```

```
CREATE TABLE learn_cassandra.users_by_country (
  country text,
  user_email text,
  age smallint,
  first_name text,
  last_name text,
  PRIMARY KEY (country, user_email)
) WITH CLUSTERING ORDER BY (user_email ASC)
AND additional_write_policy = '99p'
AND allow_auto_snapshot = true
AND bloom_filter_fp_chance = 0.01
AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
AND cdc = false
AND comment = ''
AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
AND memtable = 'default'
AND crc_check_chance = 1.0
AND default_time_to_live = 0
AND extensions = {}
AND gc_grace_seconds = 864000
AND incremental_backups = true
AND max_index_interval = 2048
AND memtable_flush_period_in_ms = 0
AND min_index_interval = 128
AND read_repair = 'BLOCKING'
AND speculative_retry = '99p';
cqlsh> |
```

Рисунок 8 – Определение стратегий для таблицы users_by_country

```
cqlsh> CREATE TABLE learn_cassandra.users_by_country_with_levelled_compaction (
... country text,
... user_email text,
... first_name text,
... last_name text,
... age smallint,
... PRIMARY KEY ((country), user_email)) WITH
... compaction = { 'class' : 'LevelledCompactionStrategy' };
cqlsh> |
```

Рисунок 9 – Создание таблицы


```
cqlsh> DESCRIBE TABLE learn_cassandra.users_by_country_with leveled_compaction;

CREATE TABLE learn_cassandra.users_by_country_with leveled_compaction (
  country text,
  user_email text,
  age smallint,
  first_name text,
  last_name text,
  PRIMARY KEY (country, user_email)
) WITH CLUSTERING ORDER BY (user_email ASC)
  AND additional_write_policy = '99p'
  AND allow_auto_snapshot = true
  AND bloom_filter_fp_chance = 0.1
  AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
  AND cdc = false
  AND comment = ''
  AND compaction = {'class': 'org.apache.cassandra.db.compaction.LeveledCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
  AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
  AND memtable = 'default'
  AND crc_check_chance = 1.0
  AND default_time_to_live = 0
  AND extensions = {}
  AND gc_grace_seconds = 864000
  AND incremental_backups = true
  AND max_index_interval = 2048
  AND memtable_flush_period_in_ms = 0
  AND min_index_interval = 128
  AND read_repair = 'BLOCKING'
  AND speculative_retry = '99p';
cqlsh> |
```

Рисунок 10 – Указание стратегии сжатия и проверка результата

```
cqlsh> CREATE TABLE learn_cassandra.users_by_country_sorted_by_age_asc (
  ... country text,
  ... user_email text,
  ... first_name text,
  ... last_name text,
  ... age smallint,
  ... PRIMARY KEY ((country), age, user_email)) WITH CLUSTERING ORDER BY (age ASC);
cqlsh> INSERT INTO learn_cassandra.users_by_country_sorted_by_age_asc (country,user_email,first_name,last_name,age)
VALUES('US','john@email.com','John','Wick',10);
cqlsh> INSERT INTO learn_cassandra.users_by_country_sorted_by_age_asc (country,user_email,first_name,last_name,age)
VALUES('UK','peter@email.com','Peter','Clark',30);
cqlsh> INSERT INTO learn_cassandra.users_by_country_sorted_by_age_asc (country,user_email,first_name,last_name,age)
VALUES('UK','bob@email.com','Bob','Sandler',20);
cqlsh> INSERT INTO learn_cassandra.users_by_country_sorted_by_age_asc (country,user_email,first_name,last_name,age)
VALUES('UK','alice@email.com','Alice','Brown',40);
cqlsh> SELECT * FROM learn_cassandra.users_by_country_sorted_by_age_asc;
```

country	age	user_email	first_name	last_name
US	10	john@email.com	John	Wick
UK	20	bob@email.com	Bob	Sandler
UK	30	peter@email.com	Peter	Clark
UK	40	alice@email.com	Alice	Brown

```
(4 rows)
cqlsh> |
```

Рисунок 11 – Создание таблицы с определением столбца кластеризации для сортировки данных с последующей вставкой информации

Вывод

В ходе выполнения работы был изучен процесс работы с распределенной базой данных Apache Cassandra: с помощью Docker Compose развернут кластер из трех узлов, создано ключевое пространство с коэффициентом репликации 2 и спроектированы таблицы с различными структурами первичных ключей. В результате практического эксперимента было установлено, что выбор ключа раздела (partition key) критически влияет на распределение данных по узлам кластера и производительность запросов: использование недостаточно селективного ключа может привести к обращению ко всем узлам, что снижает эффективность.

Также были изучены уровни согласованности (consistency levels) и их влияние на актуальность данных при операциях чтения и записи, а также рассмотрены стратегии сжатия и сортировки данных внутри таблиц. Таким образом, получены практические навыки конфигурирования распределенной базы данных и понимание принципов денормализации и проектирования схем, ориентированных на конкретные запросы, что является основой для построения масштабируемых систем.

Ответы на контрольные вопросы

1 Какой Snitch вы выбрали при создании docker-compose и почему? Какие варианты Snitch для Cassandra бывают?

При настройке кластера Cassandra в Docker был выбран GossipingPropertyFileSnitch, так как он обеспечивает автоматическое распространение информации о топологии через gossip-протокол, поддерживает работу с несколькими датацентрами (datacenter1 и datacenter2) и позволяет гибко настраивать DC/рэки через переменные окружения, что критично для динамических окружений с контейнерами. Этот Snitch минимизирует ручное вмешательство в конфигурацию и соответствует рекомендациям для продакшен-сред.

В Apache Cassandra доступны различные варианты Snitch, включая SimpleSnitch, RackInferringSnitch, GossipingPropertyFileSnitch и другие. GossipingPropertyFileSnitch является наиболее рекомендуемым, так как использует информацию о датацентрах и стойках из cassandra-rackdc.properties и учитывает динамические изменения кластера. SimpleSnitch подходит для одноцентровых развертываний, а RackInferringSnitch автоматически определяет стойки и датацентры по IP-адресам.

2 Укажите формулу уровней согласованности, которая обеспечивает высокую согласованность данных.

Формула уровней согласованности в Apache Cassandra определяется следующим образом: $R+W>N$

где:

- R — уровень согласованности для операций чтения (количество узлов, с которых должно быть получено подтверждение чтения).
- W — уровень согласованности для операций записи (количество узлов, которые должны подтвердить запись).

- N — фактор репликации (количество узлов, хранящих копию данных).

Для обеспечения высокой согласованности данных обычно выбирают кворумную стратегию: $R=W=N/2+1$ что гарантирует пересечение множеств узлов при операциях чтения и записи, обеспечивая актуальность данных.